

Lab4 Reflection

Question #1

Compare the approach used to how you would solve this in java or C++

The main difference between the Julia implementation and a similar solution in a language like C++ lies in the `describe_collision` and `area` functions. In both C++ and Julia the initial setup is mostly the same. We define a parent object `GeometricShape` and then define a couple shape objects that extend `GeometricShape`.

This is where the main differences start. In C++ the compiler only dispatches based on the first object passed to a function. This means you need to either check the type of the shapes in the function itself, likely with a long series of if else's or a switch statement. Or you need to create dedicated functions for each collision type, requiring you to know both shapes type at the time of collision. In Julia this problem is solved through the compiler's use of multiple dispatch. The Julia compiler looks at all objects passed to a function and dispatches the matching definition. If we want to add a new collision type we can just add a new function definition.

The implementation of the `area` function also differs from Julia to a language like C++. In C++ the function would likely be a method defined as part of the class. This means if we want to edit or add the function later we need to open the class up and get under the hood, modifying the class's definition. In Julia meanwhile we can just add a new function definition at any time without touching the original object definition.